School of Future Tech

Case Study Report

on

Transport Billing & Invoice Management System

by

Ayush Kumar Singh

1500967250

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Transport companies typically manage billing through manual processes such as handwritten receipts or spreadsheets. These methods are time-consuming, prone to calculation errors, and difficult to track as trip volumes grow. This case study focuses on designing and implementing a **Transport Billing & Invoice Management System** using C++ that automates the full billing lifecycle.

The system is built using core Object-Oriented Programming (OOP) principles in C++17. It features four key capabilities: ID-based selection of clients, trips, and invoices; CSV-file persistence (db_clients.csv, db_trips.csv, db_invoices.csv) so data survives program restarts; smart pointer memory management using unique_ptr, eliminating all manual new/delete; and safe deletion of clients, trips, and invoices with referential-integrity checks. It demonstrates how academic concepts like classes, vectors, file handling, smart pointers, and input validation apply to a real-world business problem.

2. Problem Statement / Case Background (Abstract)

Background

Manual billing in transport operations leads to frequent errors in fare calculation, missing payment records, no GST tracking, and difficulty retrieving past invoices. There is a clear need for a simple automated system to handle these tasks reliably and consistently.

Abstract

This case study presents the design and implementation of a **Transport Billing & Invoice Management System** using C++. The system allows operators to register clients, log transport trips, generate itemised invoices with 18% GST, process payments with change calculation, save invoices as persistent text files, and delete records safely. The system is built around four core capabilities: (1) ID-based selection — any client, trip, or invoice is chosen by its numeric ID rather than always defaulting to the last entry; (2) CSV database persistence — three files (db_clients.csv, db_trips.csv, db_invoices.csv) are saved after every change and reloaded at startup, so no data is lost between sessions; (3) Smart pointer memory management — vector<unique_ptr<T>> and make_unique<>() replace all raw new/delete, eliminating memory leaks and double-delete risk; (4) Safe deletion — clients, trips, and invoices can be deleted by ID, with referential-integrity checks that block deletion of any record still referenced by an existing invoice. It uses four core classes — Client, Trip, Invoice, and Payment — owned in STL vectors of unique_ptr. Input validation prevents crashes on bad input.

3. Case Study Design

The system is designed as a menu-driven C++ console application with CSV persistence, smart pointer ownership, and safe record deletion. Each class or function handles one responsibility:

1. **Client** — Stores client ID, name, and contact number. Includes display() to list for selection, and toCSV() / fromCSV() for file persistence.
2. **Trip** — Stores trip details and calculates subtotal and 18% GST. Includes display() to list for selection, and toCSV() for file persistence.
3. **Invoice** — Links a Client and Trip selected by ID; prints and saves the invoice to a .txt file. Includes toCSV() / fromCSV() and a second constructor for CSV reload that reconnects object graph by matching IDs.
4. **Payment** — Processes payment for any invoice chosen by ID, calculates change, and marks the invoice as PAID.
5. **getNumber()** — Helper functions for input validation; re-prompts on bad input. Includes getInt() for integer ID entry.
6. **main()** — Menu loop; owns all objects in vector<unique_ptr<T>>; loads all CSVs at startup and saves after every mutation; no manual delete needed.

7. **deleteClient()** — Lists all clients, prompts for a Client ID, and removes the record. Refuses deletion if any invoice references that client, preventing dangling pointers.
8. **deleteTrip()** — Lists all trips, prompts for a Trip ID, and removes the record. Refuses deletion if any invoice references that trip.
9. **deleteInvoice()** — Lists all invoices with their payment status, prompts for an Invoice ID, and removes the record. No referential check needed as no other records depend on invoices.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key methods and algorithms:

1. **Billing calculation**
   Subtotal = baseFare + (distanceKm x ratePerKm) + extraCharges.
   GST = Subtotal x 0.18.
   Total = Subtotal + GST.
2. **OOP concepts applied**
   Encapsulation: all data members private, accessed via const getters.
   Abstraction: billing logic encapsulated inside Trip::getTotal().
   Smart pointers: vector<unique_ptr<T>> used for all three object stores (Client, Trip, Invoice); make_unique<>() replaces all new; destructors called automatically, no manual delete loops. Invoice holds raw observer pointers (Client*, Trip*) since it observes but does not own. Payment is a lightweight stack object — not stored in a vector — and holds a raw Invoice* for the duration of one transaction.
3. **CSV persistence**

saveClients() / saveTrips() / saveInvoices() write comma-delimited rows using ofstream; each is called immediately after every change so the file is always current.

loadInvoices() reconstructs the object graph by matching stored clientId and tripId against already-loaded vectors; ID counters auto-advance past the highest saved ID to avoid collisions.

4. **ID-based selection**

   Generate Invoice: calls display() on all clients, then all trips; user enters the desired IDs; linear search on getId() locates each object; error shown if not found.

   Process Payment: lists all pending (unpaid) invoices first; user picks the invoice ID via getInt(); any invoice can be targeted directly by ID.

5. **Input validation**

   getNumber() and getInt() helpers use cin.fail() and cin.clear() to reject non-numeric input and re-prompt until valid.

   Menu choice validated with cin.fail() check before processing.

6. **Safe deletion with referential integrity (new)**

   deleteClient() and deleteTrip() scan the invoices vector before erasing; if a matching clientId or tripId is found, deletion is aborted with an error message, preventing dangling raw pointer dereferences in Invoice.

   deleteInvoice() performs a linear search by invoice ID and erases freely; the CSV is re-saved immediately after every successful deletion so the file always stays in sync.

Technology Stack Used for the Case

- I Programming language: C++ (C++17).
- I Libraries and tools:
  - <iostream> for console input and output.
  - <fstream> for writing invoice files using ofstream.
  - <vector> for dynamic storage using vector<unique_ptr<T>>.
  - <iomanip> for formatted currency output with setprecision(2).
  - <memory> for unique_ptr and make_unique.
- I Compiler: g++ with C++17 standard flag.
- I IDE: VS Code

5. Problem Statement / Case Study Implementation Details and Snapshots.

The system is implemented in a single C++ source file with nine menu-driven operations:

1. **Register New Client**

   User enters name and contact. A Client object is created with

auto-incremented ID (101+), stored in a vector<unique_ptr<Client>>, and immediately saved to db_clients.csv.

2. **Log New Transport Trip**
   Distance, rate per km, base fare, and extra charges are entered via getNumber(). A Trip object is created, stored in a vector<unique_ptr<Trip>>, and saved to db_trips.csv.
3. **Generate Invoice**
   All clients and trips are listed; user picks a Client ID and a Trip ID. An Invoice object is created via make_unique, total auto-calculated with 18% GST, printed to console, saved as Invoice_<ID>.txt, and appended to db_invoices.csv.
4. **Process Payment**
   Pending invoices are listed; user picks an invoice by ID via getInt(). Payment is processed, invoice marked PAID if sufficient, change shown if overpaid; Invoice .txt and db_invoices.csv are both re-saved to reflect the updated status.
5. **View All Invoices**

6. **Delete Client**
   All registered clients are listed; user enters a Client ID. If any invoice references that client, deletion is refused with an error. Otherwise the client is erased from the vector and db_clients.csv is immediately re-saved.
7. **Delete Trip**
   All logged trips are listed; user enters a Trip ID. If any invoice references that trip, deletion is refused. Otherwise the trip is erased and db_trips.csv is re-saved.
8. **Delete Invoice**
   All invoices are listed with their PAID/PENDING status; user enters an Invoice ID. The invoice is erased and db_invoices.csv is re-saved immediately.

Snapshots (attach below):

- Main menu screen showing all 9 options.

```
**************************************************
* Transport Billing & Invoice Management System   *
**************************************************

[DB] Loaded 1 client(s).
[DB] Loaded 1 trip(s).
[DB] Loaded 1 invoice(s).

--- MAIN MENU ---
1. Register New Client
2. Log New Transport Trip
3. Generate Invoice
4. Process Payment
5. View All Invoices
6. Delete Client
7. Delete Trip
8. Delete Invoice
9. Exit
Enter your choice:
```

- Invoice generated on console with GST breakdown.

```
========================================
              INVOICE #1002
========================================
Client Details:
Name:    Ayush
Contact: 1234567812
----------------------------------------
Trip Details (Trip ID: 501):
Distance:       100.00 km
Rate per km:   Rs.30.00
Base Fare:     Rs.120.00
Extra Charges: Rs.30.00
----------------------------------------
Subtotal:      Rs.3150.00
GST (18%):     Rs.567.00
----------------------------------------
TOTAL AMOUNT:  Rs.3717.00
STATUS:        PENDING
========================================
[INFO] Invoice saved to Invoice_1002.txt
```

- Payment processed successfully with change displayed.

```
--- MAIN MENU ---
1. Register New Client
2. Log New Transport Trip
3. Generate Invoice
4. Process Payment
5. View All Invoices
6. Delete Client
7. Delete Trip
8. Delete Invoice
9. Exit
Enter your choice: 4

Pending Invoices:
  [ID: 1002]  Rs.3717.00  -  PENDING
Enter Invoice ID to pay: 1002
Total Due: Rs.3717.00
Enter payment amount: Rs.4000

[SUCCESS] Payment of Rs.4000.00 processed successfully.
[INFO] Change to return: Rs.283.00
[INFO] Invoice saved to Invoice_1002.txt
```

- Saved Invoice_1001.txt file showing STATUS: PAID after payment.

```
==========================================
               INVOICE #1002
==========================================
Client Details:
Name:    Ayush
Contact: 1234567812
------------------------------------------
Trip Details (Trip ID: 501):
Distance:       100.00 km
Rate per km:   Rs.30.00
Base Fare:     Rs.120.00
Extra Charges: Rs.30.00
------------------------------------------
Subtotal:      Rs.3150.00
GST (18%):     Rs.567.00
------------------------------------------
TOTAL AMOUNT:  Rs.3717.00
STATUS:        PAID
==========================================
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

- I ID-based selection allows any client, trip, or invoice to be chosen precisely by its numeric ID, ensuring operations always target the intended record.
- I CSV persistence ensures all clients, trips, and invoices are reloaded on every program start; no data is lost between sessions, and the .txt invoice file always reflects the true payment status.
- I Smart pointer ownership with vector<unique_ptr<T>> and make_unique<>() eliminates all manual memory management; GST calculation remains accurate and the invoice breakdown clearly separates subtotal, GST, and total.
- I Safe deletion with referential integrity prevents orphaned references: clients and trips cannot be removed while an invoice depends on them, protecting the raw observer pointers held by Invoice from ever becoming dangling.

Conclusion

This case study demonstrates a complete C++ implementation of a Transport Billing and Invoice Management System. The system provides full record management: clients, trips, and invoices are selected by numeric ID; all data persists across sessions in CSV files loaded at startup and saved on every change; smart pointers (unique_ptr / make_unique) replace all manual new/delete, ensuring safe memory management; and clients, trips, and invoices can be deleted safely, with referential-integrity checks blocking any deletion that would leave an invoice with a dangling pointer. The system integrates client registration, trip logging, billing calculation with GST, invoice file handling, payment tracking, and safe record deletion into a coherent console application, providing a strong foundation for further development with a relational database or a graphical interface.

7. References

I Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.

I cppreference.com — Official C++ standard library documentation including iostream, fstream, and vector.

I GeeksforGeeks — Tutorials on C++ OOP, file handling, vectors, and input validation.

I Government of India — GST Rate Structure, Ministry of Finance (https://www.gst.gov.in).

I ITM Skills University — B.Tech CSE Semester II Case Study Guidelines, 2025.